Liu *et al. Cybersecurity* (2025) 8:32
https://doi.org/10.1186/s42400-024-00335-4

Cybersecurity

RESEARCH Open Access

LLM4TDG: test-driven generation of large language models based on enhanced constraint reasoning

Jingqiang Liu[1,2], Ruigang Liang[1], Xiaoxi Zhu[1], Yue Zhang[1,2], Yuling Liu[1,2] and Qixu Liu[1*]

Abstract

With the evolution of modern software development paradigms, component reuse, and low-code approaches have emerged as mainstream in software development. However, developers often lack an in-depth understanding of reused code. The inability of components to operate autonomously leads to insufficient testing of software functionalities and security, further exacerbating the contradiction between the increasing complexity of software architectures and the demand for accurate and efficient software automation testing. This, in turn, increases the frequency of software supply chain security incidents. This paper proposes a test-driven generation framework, LLM4TDG, based on large language models (LLMs). By formally defining the constraint dependency graph and converting it into context constraints, LLMs' ability to understand natural language descriptions such as test requirements and documents is enhanced. Constraint reasoning and backtracking mechanisms are then used to generate test drivers that satisfy the defined constraints automatically. Using the EvalPlus dataset, we evaluate the comprehensive capabilities of LLM4TDG in test case generation using four general-domain LLMs and five code-generation-domain LLMs. The experimental results indicate that our approach significantly enhances LLMs' ability to comprehend constraints in testing objectives, achieving a 47.62% increase in constraint understanding across 147 testing tasks. Employing LLM4TDG significantly improves the average pass@k metric of all LLMs by 10.41%. The pass@k metric for CodeQwen-chat has improved by up to 18.66%. The metric surpasses the state-of-the-art GPT-4, with a performance of 92.16% on HUMANEVAL and 87.14% on HUMANEVAL+, which enhances the error correction and functional correctness in test-driven code generation. Meanwhile, Our experiments were conducted on a dataset of Python third-party libraries containing malicious behavior in the context of security testing tasks, validating the effectiveness of our method in real-world applications and its generalization capabilities.

Keywords Large language model, Test driven development, Software development, Software supply chain security

Introduction

The development model of modern software does not involve writing code from scratch; instead, it accelerates software development and reduces costs by reusing third-party components. However, this model poses significant vulnerabilities in the software supply chain, primarily due to inadequate security testing of dependency libraries, which has led to new security crises in software development in recent years. The frequent occurrence of software supply chain security incidents globally has not only posed a severe threat to network security but also

*Correspondence:
Qixu Liu
liuqixu@iie.ac.cn
[1] Institute of Information Engineering, Chinese Academy of Sciences, 100085 Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, 100049 Beijing, China

Springer Open

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

impacted the property, privacy, and even national security of daily work and life. Software dependency library attacks have become a favored target primarily due to their low cost and high efficiency. On the one hand, software development relies excessively on dependency libraries stored on third-party platforms. However, these platforms have limited capabilities in detecting malicious code and vulnerabilities in dependency libraries, making it easy for malicious dependency libraries to be published on these platforms. On the other hand, due to the chain structure of the software supply chain, security issues caused by attacks targeting upstream vulnerabilities can continuously propagate and spread to downstream links, significantly expanding the attack scope.

Microsoft developers identified a typical software supply chain attack involving obfuscated malicious code in the dependency library of the xz tool (2024), revealing the threat posed by supply chain vulnerabilities to network security. Through continuous monitoring of the statistical data of mainstream open-source software repositories across the network (2024), the Python Package Index (PyPI) repository for Python has become a prime target for attackers. According to a report released by Sonatype (2022), developers download 1.2 billion vulnerable dependency components every month, and approximately six out of every seven projects with vulnerabilities are affected by transitive dependencies. The Open Source Security and Risk Analysis (OSSRA) report (2023) points out that 91% of code repositories contain components that have not been updated in two years. In the software development process, software testing is crucial for ensuring code quality and stability. With the increasing scale and complexity of software, greater demands are placed on testing the capabilities of dependency libraries. However, the difficulty of testing dependency libraries is their inability to run automatically. Static analysis is often used for dependency library testing, but it has inherent issues such as a high false positive rate, and the interpretation of detection results relies heavily on expert experience. The manual approach of writing test drivers to achieve dynamic testing of dependency libraries suffers from high development costs and low testing efficiency. Automated test case generation methods have become an important technical means to improve efficiency, but current methods often lack universality and face issues such as low compilation pass rates and poor accuracy.

Automatically generating programs that accurately correspond to user intentions has long been a challenge in the computer science field (Gulwani et al. 2017). Over the past few decades, classical program synthesis techniques have emerged, including deductive synthesis (Green et al. 1981), inductive synthesis (Gulwani et al. 2011), and neural-guided synthesis (Kalyan et al. 2018). Currently, most program synthesis techniques involve processing expressions, transforming them into constraint problems, and solving them using satisfiability modulo theories (SMT) solvers. With the advent of large models (Brownet et al. 2020), researchers have leveraged the natural language understanding and generation capabilities of large language models (LLMs) to explore their potential in code generation (Austin et al. 2021). Approaches such as CODEX (Chen et al. 2021) and CodeGen (Nijkamp et al. 2023) predict the next token autoregressively, generating code snippets based on function signatures and program function descriptions to synthesize program code that meets user intentions. Although Large Language Models (LLMs) possess a specific capability for code generation, they still face challenges with accuracy, making it difficult to apply them effectively in real-world scenarios. Consequently, there remains room for improvement in the correctness of test case generation by LLMs.

In this paper, we propose a test-driven generation framework, LLM4TDG, based on LLMs that utilizes constraint dependency graphs to enhance semantic information, employs constraint reasoning and backtracking mechanisms, and enhances LLMs' ability to understand contextual constraint information. The goal is to enhance LLMs' understanding of test requirements, automate the construction of test constraints and reasoning, and generate test case code that meets constraint requirements, thereby reducing reliance on expert experience and improving the correctness and automation of test-driven generation.

Contributions. Our main contributions are outlined below:

- We propose a novel approach for representing testing requirements through constraint dependency graphs. By formalizing the definition of syntactic and semantic constraints and constructing constraint dependency graphs accordingly, this methodology notably augments the capacity of LLMs to comprehend the requirements of testing tasks.
- We design an LLM-based test-driven generation framework, which automatically understands and transforms the given test task requirements, application programming interface (API), documentation, and other input descriptions into contextual constraints, enabling the generation of test case code that meets the constraint requirements.
- By employing constraint reasoning and a backtracking mechanism, we iteratively optimize test drivers based on runtime feedback. This mechanism improves the capability of code error repair and functional correctness of LLM test-driven generation, which surpassed that of the state-of-the-art (SOTA)

LLMs. Meanwhile, the effectiveness of our method in terms of its applicability and generalization capabilities was validated in the context of security testing tasks.

Related work

Model-driven testing

Model-driven testing utilizes models to describe the system's behavior under test (SUT) and generate test cases, effectively achieving software automation testing. Finite state machines (Wagner et al. 2006) are significant in various fields, such as hardware design, speech recognition, protocol conformance testing, and object-oriented testing, as they receive inputs from applications and produce actions that affect the applications. Minj et al. (2015) created a node-linked list based on the Stateflow model, converting it into a finite state machine model to generate test cases. The Object Management Group (OMG) proposed a methodology and framework specification Model Driven Architecture (MDA) (Mussa et al. 2009) for model-driven software system development, which includes a platform independent model (PIM) and a platform specific model (PSM). MDA separates the functional description of the system from its implementation, significantly addressing issues of low software development efficiency and reusability (Hsu et al. 2014). With the release of the Unified Modeling Language (UML) 2.0 Testing Profile, model-driven testing has gradually shifted from an initial theoretical exploration phase to a stage of development and practice as an automated testing method. In model-driven software testing, only the construction of the test model and the definition of model transformation need to be focused on, while subsequent testing tasks are automatically performed by computers. Also, it allows testers to concentrate on the development of test models, improving the level of automation in software testing. Wang et al. (2005) suggest that model-driven testing can build comprehensive and precise test models by analyzing the software requirements of the SUT and combining test strategies, enabling the automatic generation of test cases.

The primary advantage of model-based testing lies in its ability to achieve a high level of automation in the software testing process. Instead of requiring testers to manually write test cases, an abstract model of the SUT combined with testing tools is utilized to generate test cases from the model automatically. PIMs can be directly used in the development process for test case generation, where modeling languages are employed to establish test requirements directly based on the system design model. Simultaneously, the PIMs are transformed into test models. The AutoTCG platform employs a graphical model-based testing engine and advanced algorithms, providing various test models such as cause-effect graph models and combinatorial pairing models. The platform allows for the selection of different models to complete test design, facilitating efficient and convenient automated generation of test cases. Moreover, model-based testing can generate different sets of test cases based on the choice of different test coverage strategies, enabling in-depth testing or smoke testing of specific features of the SUT, thus covering the intended functionality. This testing methodology can even meet test requirements (Schneider. 2021), trace them automatically back to the test model, and offer other test coverage reports. The FuzzGPT (2023) initializes the LLM to generate test case programs for fuzz testing through fine-tuning and contextual learning of LLMs, achieving fully automated testing and discovering the potential of leveraging ChatGPT's command-following ability for fuzz testing. (Zhang C. et al. 2023) explored the effectiveness of automatically generating fuzz drivers based on LLMs by leveraging their code understanding and generation capabilities. The results showed that the test drivers generated based on LLMs achieved a 64% accuracy rate, demonstrating initial practical utility. (Lyu Y. et al. 2023) proposed a PromptFuzz method that utilizes LLM prompts to guide data mutation and iteratively generates fuzz drivers, achieving higher branch coverage.

Data-driven testing

Data-driven testing (Baliga et al. 2017) decouples test data from test scripts by defining test case templates that read data from data files, thereby reducing the coupling between data and code. Random test data generation methods can be blind, leading to the redundant coverage of specific state spaces while others remain uncovered. Data mutation provides an effective way to enhance the adequacy of test sets, yet the efficient generation of mutated test data remains an urgent issue that needs to be addressed. To this end, numerous scholars have conducted research on the generation of mutated test data, primarily including constraint-based methods, coverage methods based on data flow and control flow (Offutt et al. 1999), hybrid methods (Nardo et al. 2015), and search-based methods (Carlos et al. 2014). Constraint reasoning research primarily focuses on constraint search and constraint languages. Among them, constraint search mainly investigates constraint satisfaction over finite domains, such as backtracking, constraint propagation, intelligent backtracking, and truth maintenance. Symbolic execution methods (King et al. 1976) use symbolic variables to traverse programs and calculate path constraints through constraint solvers. One of its challenges lies in solving complex constraints involving nonlinear

and floating-point numbers. Early symbolic execution, essentially a static analysis of programs, could not handle dynamic structures such as array indices and suffered from path explosion issues. Dynamic execution methods obtain runtime information by executing programs. Using specific values for symbolic execution simplifies the constraint system and improves the possibility of solving complex constraints. However, this approach can blindly reduce the solution space (Kannavara et al. 2015). Liu et al. (2011) employed a dynamic symbolic execution approach, replacing program inputs with symbolic variables, and proposed a mutation data generation method with dynamic domain reduction. The generated test data were able to detect 98% of the mutants.

To efficiently generate test data that satisfy structural coverage criteria, search-based methods have become a research hotspot in recent years (Yao et al. 2016). Heuristic search algorithms, which often mimic natural phenomena, transform the constraint satisfaction problem (CSP) into finding optimal solutions in a search space. Compared to the blind search of random methods, heuristic search leverages existing information to guide the next search, thereby improving the search efficiency. Notable examples include genetic algorithms (Pargas et al. 1999) and ant colony optimization (López-Ibáñez et al. 2015). The use of evolutionary algorithms (Rani et al. 2019) to generate test data is a significant research direction in software testing. Specifically, evolutionary genetic algorithms have been adopted to design various multi-population genetic operations for generating test data for mutation-branch-driven problems. These algorithms do not require the optimized objective function to be continuous or differentiable, and they can find solutions to complex problems within a reasonable time frame. However, heuristic search methods are often overly dependent on factors such as the initial search point, evaluation function design, and search space limitations, often requiring multiple iterations. Hybrid execution methods, which combine symbolic execution and heuristic search methods, can effectively handle complex constraints involving nonlinearities and floating-point numbers by executing programs with symbolic variables to obtain path constraints and then searching for test data using heuristic search methods.

Requirement-driven testing

Requirement-driven testing focuses on the implementation and evaluation of testing activities around the requirements of software products, emphasizing the close relationship between testing activities and software requirements. The testing process begins with a thorough understanding and analysis of software requirements to ensure that testing comprehensively covers all functions and features defined in the requirements. ASSERT (Moitra et al. 2019) starts from the Requirement Catch Environment (RCE), which allows engineers to write requirements in a human-readable and machine-analyzable manner. It utilizes a Requirement Analysis Engine (RAE) to analyze requirements, eliminating conflicts, incompleteness, and other errors, and automatically generates test objectives and effective test cases based on testing strategies. Yang et al. (2014) combined model-driven concepts and defined a metamodel and modeling method for test requirements. They utilize this method to obtain a test requirement model, derive corresponding test objectives, and generate corresponding test cases. Aniculaesei et al. (2018) proposed a method for automatically generating test cases using the NuSMV model checker, considering the requirement-based coverage criteria Requirement Coverage (RC), Antecedent Coverage (AC), and Unique First Coverage (UFC) proposed by Whalen et al. (2006). They evaluated the quality of test cases through mutation testing. Aniculaesi et al. (2019) integrated this method into the SCADE toolchain as a model checker and conducted experiments in an automatic door-locking system for vehicles, achieving good testing results. With technological advancements, behavior-driven development (BDD) has transformed test cases from a technical level to a business level. It uses natural language to write test cases and defines standardized grammatical structures to describe the behavior of the SUT, improving the readability and maintainability of test cases.

In terms of quality assessment for the generation of test case code, when evaluating the correctness of code produced by LLMs, traditional natural language processing (NLP) metrics such as the BLEU score (Papineni et al. 2002) are no longer reliable in the context of program synthesis, as two distinct code fragments can be semantically equivalent. Methods such as translation validation (Bang et al. 2022) for verifying problems in specific domains are already challenging enough, let alone constructing a universal verifier with absolute certainty to prove the correctness of arbitrary problems, including those in code benchmarks. Code benchmarks like HUMANEVAL rely heavily on manually constructed test cases to evaluate LLM solutions. These tests tend to be relatively simple and fail to explore the functionality or boundary cases of the code comprehensively. The Python-centric dataset MBPP is created by crowdsourced participants, where each problem consists of a problem statement, a function signature, and three test cases. Additionally, there are other benchmarks for other languages, such as Spider (Yu et al. 2018) (SQL), HUMANEVAL-X (Zheng et al. 2023) (C++, Javascript, and Go), CodeContests (Li et al. 2022) (C++ and Java),

and MultiPL-E (Cassano et al. 2023), which extend HUMANEVAL and MBPP to 18 programming languages. Recently, researchers have created more realistic code evaluation benchmarks by collecting GitHub issues and corresponding code repositories to measure the ability of LLMs to perform real-world tasks (Jimenez et al. 2023).

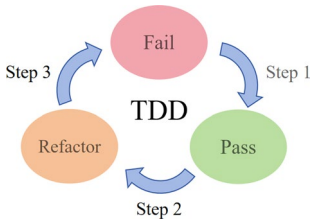


Fig. 1 The TDD paradigm of "Red Green Refactor"

Preliminaries

Test driven development

Software testing is a crucial means to ensure the quality and reliability of software, as well as a commonly used method to check the correctness of software functions. Software quality assurance methods include formal proofs and software testing. Formal proofs (Eichenbaum et al. 1965) refer to the use of rigorous mathematical reasoning to demonstrate the correctness of software. Theoretically, formal proofs are more reliable than software testing, but they are difficult to implement, thus making software testing a common technical approach to ensure the correct execution of software. However, software testing often accounts for 30% to 50% of the software development cycle, representing a significant time cost.

Under the premise of ensuring software quality, improving the efficiency of software testing has become a research focus. The key to software testing lies in the acquisition of test data. Manually designing test data is inefficient and prone to errors and incomplete considerations. Automated testing techniques, including white-box testing, black-box testing, and gray-box testing methods, can shorten testing time and reduce costs. They are becoming widely used methods for identifying software security issues. White-box methods provide better-quality test cases by analyzing the source code of the SUT. For instance, symbolic execution generates tests for deep paths by solving symbolic path constraints, thereby breaking coverage platforms. Black-box testing methods provide inputs to the SUT without knowing the program's internal structure. Traditional black-box testing techniques are primarily divided into generative-based testing (Holler et al. 2021) and mutation-based testing (Winterer et al. 2020). Gray-box testing methods focus not only on program input and output but also on the program's internal structure, often using coverage information from the SUT as feedback to mutate input data, such as coverage-guided fuzz testing methods. However, the aforementioned methods are unsuitable for generating tests with semantically constrained inputs, and the current development of general software automated testing techniques is still immature. The primary challenge lies in the inability to guarantee the correctness of the test program.

With the rise of agile development models, test-driven development (TDD) has become an essential component of Scrum, continuous integration, and other methodologies. The "Red-Green-Refactor" TDD paradigm is illustrated in Fig. 1. Here, "Red and Green" originate from the unit testing framework, where a failing test is indicated in red, and a passing test is shown in green. Red represents the state where a new test has been written but has not yet passed; green signifies that the functional code has been written and the test has passed. Refactoring, on the other hand, is the process of adjusting the code after the basic functionality has been completed. Refactoring is a crucial aspect of TDD. The second step, "Green", only focuses on passing the test, refactoring returns to the test case code itself, and addressing semantic and logical imperfections in the code. In TDD, refactoring and testing complement each other. Without testing, refactoring can become aimless, and without refactoring, code chaos can increase, leading to poorer testing results. As software scales and becomes more complex in design and development, efficient and accurate test case generation, improving test coverage, and avoiding state space explosion become paramount.

LLMs for code

In recent years, artificial intelligence (AI) has made significant progress, and the introduction of deep learning algorithms such as convolutional neural networks (CNN), recurrent neural network (RNN), long short-term memory (LSTM), as well as transfer learning and evolutionary algorithms, has achieved remarkable results in specific tasks across various fields, including face recognition, autonomous driving, and speech recognition. However, research on automatic code generation analysis and code repair is still in the process of making breakthroughs. The DeepMind has developed the Neural Programmer-Interpreter, which can learn and edit simple programs independently, and its generalization ability in sorting surpasses that of sequence-to-sequence LSTM. Google's code bug prediction system, which utilizes machine learning and statistical learning to determine whether code contains defects, understands not only natural language but also constructs complex adaptive

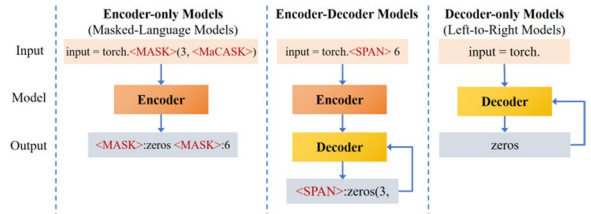


Fig. 2 Overview of different LLM architectures

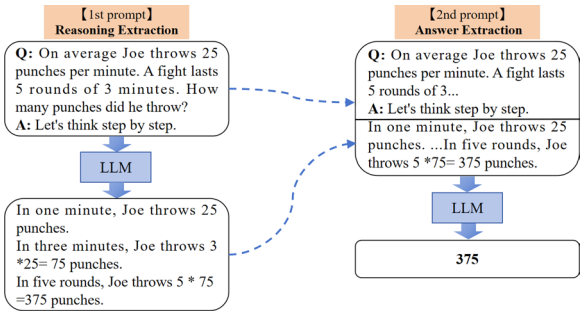


Fig. 3 Thinking chain prompts guide LLMs in semantic understanding

computer programs. Although this system has not yet achieved generalized automatic code writing, it represents a step forward in this research direction.

The emergence of generative pre-trained transformer (GPT) models, LLM Meta AI (Llama), and the Claude large model have provided novel insights for addressing existing software code generation issues. Due to their robust analytical capabilities for complex data, LLMs have demonstrated advanced performance in code-related tasks, including code generation, program repair (Xia et al. 2023), and automated testing (Xia et al. 2024). The GPT (Radford et al. 2018) model, proposed by OpenAI, aims to address a key challenge in general NLP: how to generate natural and realistic text. Renowned LLMs such as CODEX, CodeGen, INCODER (Fried et al. 2023), and PolyCoder (Xu et al. 2022) have been extensively applied to code generation, generating code snippets based on functional requirements described in natural language. These large models are essentially language models built on the Transformer architecture, encompassing encoder-type, decoder-type, and their hybrid versions, as shown in Fig. 2. By learning rich semantic features from a given word sequence within its context, they exhibit astonishing comprehension and expression capabilities in handling complex tasks and reduce the cost of reinforcement learning with human feedback (RLHF) in automated labeling. Unlike traditional RNN models (Elman et al. 1990), the Transformer model utilizes a self-attention mechanism, enabling better handling of long sequences and parallel computation, thus resulting in superior efficiency and performance. Intelligent code assistants based on LLMs, such as Devin AI, Copilot, IBM Watsonx, and Amazon CodeWhisperer (Divakaran et al. 2024), are capable of extracting deep-level features from software and possess a certain degree of code automation generation and repair, potentially revolutionizing existing software development and testing paradigms.

Chain-of-thought prompt

Chain-of-Thought Prompt is a strategy for interacting with LLMs when solving complex reasoning tasks to improve the model's reasoning performance on such tasks. As illustrated in Fig. 3, the construction of a chain of thought utilizes prompt engineering to guide the model in simulating human thought processes for logical reasoning and decision-making. The core idea is to form a logical chain through a series of reasoning steps, helping the model better understand and solve problems. This typically involves processes such as in-depth analysis of the problem, feature extraction, relational reasoning, and conclusion derivation. In scenarios requiring deep understanding and reasoning, this method guides the model to generate detailed answers that contain intermediate reasoning steps by constructing prompts for gradual reasoning steps. This process involves presenting a problem to the model, followed by a series of logically coherent reasoning steps as prompts. This emphasizes the transparency and logicality of the solution steps rather than relying solely on the model to intuitively "guess" the answer. By exhibiting the thought process of problem-solving, the model is able to handle complex issues more accurately.

For prompt engineering, the focus lies in addressing complex semantic understanding issues. In constructing a chain of thought, the model needs to learn how to identify critical information in the problem, establish connections between the information, and gradually derive the correct answer. First, the model must comprehend the context of the given input or question, effectively capturing and understanding long-distance dependencies through its multi-layer structure, including analyzing the key vocabulary, concepts, and their relationships within the problem. Subsequently, based on understanding the question, the model searches for relevant information or knowledge within the training data. This process is not traditional retrieval but rather activated by neural network pathways related to the problem. The model then ranks, compares, and synthesizes the information to form a coherent and logically sound response tailored to the question's requirements. Finally, the model generates a response based on the previous steps, involving the

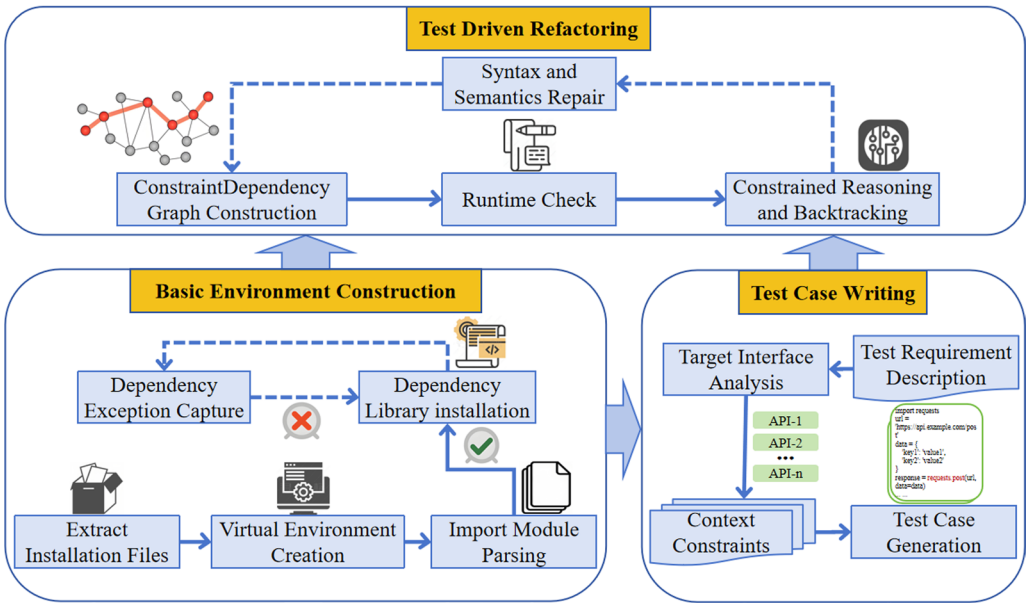


Fig. 4 A test-driven generation framework based on constraint reasoning using LLMs

selection of words and sentence structures to express the reasoning results clearly and accurately. LLMs optimize this process through language patterns and structures learned through training, ensuring that the generated text adheres to linguistic rules while also resembling human expressions.

Framework

Overview

Figure 4 shows the framework overview of LLM4TDG (LLM for test-driven generation), which automates the generation of test-driven instances by enhancing LLM's ability to understand code-testing requirements. The framework workflow comprises three parts: fundamental environment analysis, test case writing, and test-driven reconstruction. First, the basic environment analysis module acquires the runtime requirements for the test case code, parses imported dependency packages and installation files and constructs a virtual runtime environment. Second, by leveraging LLM's capabilities in code semantic understanding and intrinsic feature correlation, it comprehends code contextual constraints, function names, parameter lists, return value descriptions in interface documentation, and other testing requirements. It then analyzes and extracts syntactic and semantic constraints to generate test case code. Subsequently, a constraint dependency graph is constructed using syntactic and semantic constraints, which are traversed for parsing, reasoning, and backtracking. Moreover, a runtime-based code repair mechanism provides real-time feedback on the runtime status during test case execution. This mechanism iteratively repairs syntactic and semantic errors in the test case code, ultimately generating a test driver that satisfies the constraint requirements.

Basic environmental analysis

Virtual environment creation

The code has specific requirements for the version of the runtime environment, and the third-party modules on which the code depends also have requirements for the runtime environment version. To ensure that the test driver runs in a stable and controllable environment, it is necessary to construct a Python-based environment that meets the corresponding requirements. When calling dependent libraries and APIs, users typically follow the installation instructions in the user documentation, which also provides the version requirements for the modules and runtime environment. Furthermore, due to the differences in runtime requirements among different project codes, setting up an isolated environment ensures that each project runs in an independent and clean environment, free from the influence of other projects.

When creating a virtual isolated environment, an independent directory is generated, which contains all the executable files and dependent library modules required for a specific runtime environment. No additional files are installed in the operating system environment, thus not affecting the environment variables of the operating system. We adopt virtualenv for environment creation. virtualenv is a tool that isolates Python runtime environments, enabling easy creation, maintenance, and deletion of virtual environments on systems such as Windows

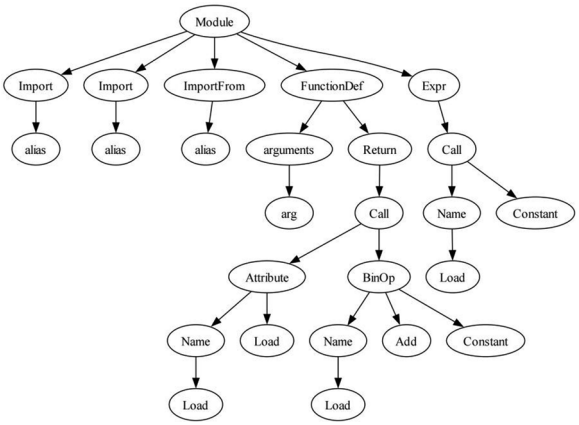


Fig. 5 Using AST to find import statements

and Linux. It allows the installation of specified versions of packages in projects without being affected by other projects or global Python environment variables. It is very useful for managing different projects, avoiding version conflicts, and maintaining the independence of project environments. When executing the virtualenv command, an instance of the Python interpreter is created, and all its configurations are saved in the.env file. After the environment is set up, it can be easily packaged and migrated to other environments for execution, simplifying the deployment process. Additionally, some dependent libraries have requirements for the operating system type, so the framework adopts both Windows and Linux operating systems to meet the particular needs of different dependent libraries.

Import module parsing

The use of Python package management tools enables the parsing of dependency library installation scripts, allowing for automatic downloading, compilation, and installation of dependent libraries from PyPI. However, the provision of dependency library installation scripts is not mandatory, and when such scripts are absent, installation may fail due to the lack of necessary module installation information. To address this issue, this paper employs a method based on the Abstract Syntax Tree (AST) to resolve the challenges encountered during the automated installation of dependent packages. ASTs are abstract representations of the syntactical structure of source code in a tree-like format. ASTs contain definitions of Expr syntax and Stmt statement abstract grammars. By parsing the abstract grammars of ASTs, we can locate all the import statements for module imports, and retrieve imported module information based on inheritance relationships, Fig. 5

An expression is a code snippet that computes a value, consisting of variable references, numerical calculations, function calls, etc. Expressions typically yield a value that can be used to form more complex expressions or assigned to a variable. A statement is an instruction that performs an operation or accomplishes an action and is used to organize the execution order of code and ensure that the program executes according to the intended logic, such as assignments, conditional judgments, and loops. Source code is parsed into an AST by reading the code file string, and the AST is traversed recursively. The traversal process searches for assignment statements of the Stmt type and checks for the presence of keywords of the Import or ImportFrom type. If present, the values of the assignment expressions are parsed as a list of dependent modules. Solutions will be formulated using LLMs for installation failures caused by issues such as network issues or version conflicts. Additionally, as some installation errors are not caused by environmental variable issues or missing dependent modules but rather by operating system-level dependencies, we set an upper limit for repeated installations to exclude cases where installation consistently fails.

Test case writing

Contextual constraint understanding

Although LLMs can adhere to programming syntax and semantics, they do not always generate functionally correct code precisely according to the requirements. Therefore, this paper proposes a method for understanding the description of the test case requirements and adding contextual constraints by constructing a constraint dependency graph based on test case code to represent the constraint dependencies in testing tasks. This approach aims to guide LLMs in generating code that meets testing needs, reduce the risk of "hallucinations" produced by LLMs, and enhance the accuracy of the model's constrained reasoning. A constraint is a relational expression containing several variables that can describe the properties, interrelations, task requirements, and objectives of domain objects. A CSP comprises a set of variables and constraints between them, representing the conditions that the variables must satisfy. It preserves structural information about the problem, such as the size of the variable domain and the correlation between variables, providing heuristic information for problem-solving. The value domain of variables may be finite; for example, a boolean variable has a value domain containing only true and false, or it may be discrete and infinite, such as the domain of integers, or continuous, such as the domain of real numbers. The goal of solving a CSP is to find the corresponding value domain for each variable so that all constraints are satisfied. Designing a constraint satisfaction system is a difficult and complex task because

Table 1 Prompt of Constraint Instruction Optimization

Instruction	Prompt Optimization
Format	**A.** The function signature lists are [$api].
	B. Extract the call and return value from the comments, and output the JSON format. The field names in JSON
	list are function_call and return_value. The value of function_call contains the actual parameters of the function.
Syntax	**A.** The Python library is $library.
	B. The Python library module names are [$module].
	C. Fix the program syntax errors .
Semantic	**A.** The code functional requirements are [$requirement].
	B. The semantic constraints are [semantic].
	C. Fix the program return value errors meet the semantic constraints.

CSPs are generally NP-hard problems. The Boolean satisfiability (SAT) problem of propositional logic formulas involves determining whether there are some values that make a given propositional formula composed of AND, OR, NOT, and variables valid.

By leveraging constraint instructions and thought chains, LLMs can be guided to understand natural language descriptions, transform them into formalized constraints in SAT format, and construct constraint dependency graphs, thereby enhancing their reasoning and analysis capabilities. Constraint instructions include formatting, syntactical, and semantic constraints, as shown in Table 1. Formatting constraints regulate the output format of LLMs by parsing the code to extract information such as function names, parameters, and return values. Syntactical constraints define content that follows the syntactic specifications of code interpreters, such as missing dependency libraries and modules and syntactical error descriptions. Semantic constraints are formalized constraints derived from LLMs' understanding of test requirements, API documentation, input/output return values, and other content. LLMs can understand constraints on function parameters and requirements for invocation rules, data types, value ranges, and formats, providing data support for constructing constraint dependency graphs.

Test code generation

The test case template can guide LLM in generating structured, well-formatted, and maintainable code. By providing the model with specific contextual semantic information, the LLM can be instructed to generate test cases that meet the expected input format and specification requirements. This provides code analysis support for constraint dependency graph construction, constraint reasoning, and test-driven code refactoring. Figure 6 illustrates the process of generating general test code through

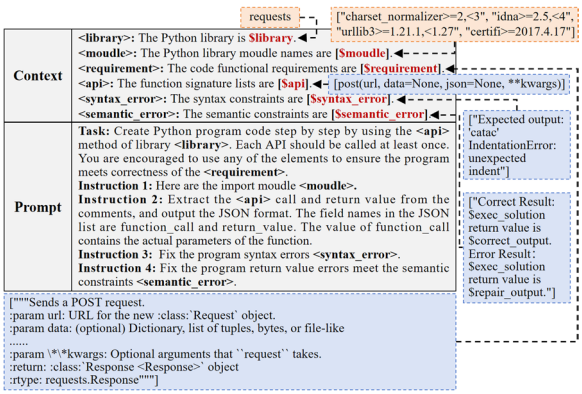


Fig. 6 Optimize prompts to generate universal test cases

LLM prompts. This process comprises two key components: contextual constraints and task prompts. Among them, the contextual constraint component provides a contextual understanding of API usage to assist the LLM in accomplishing the specified tasks, including information such as dependent libraries, modules, task requirements, function signatures, syntactic and semantic errors, etc. The task prompt component defines the steps to guide the LLM in completing the desired tasks, including descriptions of task objectives, formatting instructions, and syntactic and semantic correction instructions.

To enhance the universality of the test case template, we name variables in a symbolic form and replace variable identifiers for different testing tasks without modifying other template contents. This approach enables the LLM to understand the meaning of instructions while facilitating automated program processing. Within the contextual semantic constraints, we defined six types of constraint information, including $< library >$, $< module >$, $< requirement >$, $< api >$, $< syntax_error >$, and $< semantic_error >$. Specifically, $< library >$ describes the name of the dependency library to which the test case

API belongs, < *module*> indicates the module name and version requirements for calling the dependency library and API imports, and < *requirement* > represents the requirement descriptions obtained from test requirement specifications and interface documents, including parameter lists, return value types, and other information. < *api* > is a list of function signatures that contains the parameter lists of the called functions. < *syntax_error* > and < *semantic_error* > define the syntactic and semantic errors that occur during the dynamic execution of test cases. The prompt words include information about the test case objectives, imported modules, formatted function signatures, and descriptions of syntax and semantic error correction schemes that may arise during the execution of the test case. By utilizing the prompt words to execute operational instructions, we guide the LLM in generating standardized test case code, thereby enhancing the correctness and automation of test case execution.

Test driven refactoring

While LLMs possess the capability of code generation, they lack the pre-training and fine-tuning processes specifically tailored for test-driven generation tasks, often resulting in syntactic and semantic errors when running test case code directly, which fails to meet testing requirements. Additionally, test case code involves the combined invocation of interfaces and dependencies among multiple variables, where a change in a single variable can trigger corresponding changes in other related variables. Most existing program analysis methods model dependencies at the granularity of classes, functions, and statements, and while some methods model and analyze variable dependencies, they fail to consider the impact of contextual constraint dependencies on variables. Therefore, studying and analyzing the dependencies among variables and deepening LLMs' understanding of constraint information for dependent variables are important for improving variable constraint analysis capabilities and reducing the cost of variable satisfiability testing.

We propose a large-model constraint reasoning analysis method based on variable constraint dependency graphs, leveraging LLMs in program analysis techniques. First, we utilize LLMs to understand test requirements, API documentation, syntax, and semantic content, extract constraint information; and formally define variable constraints. Second, we analyze the test case code, extract constraint dependencies among variables, and construct constraint dependency graphs to represent the syntactic and semantic constraint information of the variables. Then, based on the syntactic and semantic constraints and the input–output feedback from the dynamic execution of test code, we parse the dependency graph to locate the problem points and backtrack to the correct locations. Finally, we reconstruct the test case code based on the problem locations and constraint requirements, generating a test driver that meets the requirements.

Constraint dependency graph construction

The variable constraint dependency graph is a direct representation of the propagation of variable constraint information in a program. It can intuitively map the constraint dependencies among variables and their propagation directions while reflecting the dependency situations of variables in the program, such as data dependencies, control dependencies, function dependencies, and other information. By traversing the variable constraint dependency graph, we can track the usage of variables throughout the program and their constraint dependencies, providing support for LLM variable constraint reasoning and the generation of test drivers that meet syntactic and semantic constraint requirements.

(1) Formal Definition

Variable Constraint Dependency (VCD): Let $VDC = (V, E, C)$, where V is the set of all variables in the test driver program P, $V = \{v_1, v_2, ..., v_n\}$; E is the set of dependency relations among variables, $E = \{e_{ij} = (v_i, v_j) \mid v_i, v_j \in V\}$; and C is the set of constraints on dependency relations, $C = \{c_k = (e_{ij}, c_k) \mid e_{ij} \in E, c_k \in C\}$, as shown in Eq. 1.

Within this, e_{ij} is defined as the directed edge representing a dependency relation from variable v_i to variable v_j.

$$e_{ij} = \begin{cases} dd,\ v_i \text{ data depends on } v_j \\ cd,\ v_i \text{ control depends on } v_j \\ fd,\ v_i \text{ function depends on } v_j \end{cases}. \tag{1}$$

Data Dependency: Let s be an assignment statement in P, with v_i being the variable assigned in s and Ref(s) being the set of variables used in s. If there exists a variable $v_j \in$ Ref(s), then variable v_i is said to have a data dependency on variable v_j, denoted as dd(v_i, v_j).

Control Dependency: Let s_1 and s_2 be an assignment statement and a control statement in P, respectively. Let v_i be the variable assigned in s_1 and v_j be the variable that controls the condition in s_2. If the execution of s_1 is determined by the execution state of s_2, then variable v_i is said to have a control dependency on variable v_j, denoted as cd(v_i, v_j).

Function Dependency: If the value of variable v depends on the return value of function F, then variable v is said to have a function dependency on F, denoted as fd(v, F).

(2) Generation of Constraint Dependency Graph

An AST encapsulates all the information of a program and represents the source code in a tree-like structure, with each node in the tree corresponding to a part of the source code. By parsing and traversing the nodes in the AST, we

extract control flow and data flow information, identify dependencies between variables based on the keyword attributes in the nodes, and add relevant variable constraints to the constraint dependency graph. Specifically, the `label` method analyzes and determines the type of each node; if the node represents an "Assign" operation, the `label` is set to "dd"; if it is a loop control type such as "For", "While", or a branch control keyword "If", the `label` is "cd"; for function calls of type "Call", the `label` is "fd". The `addLLMConstr` method collects variable constraints and adds them to the constraint set on the edges of the dependency graph.

Algorithm 1 describes the process of parsing the test driver code through the AST, extracting variable dependencies, and building the constraint dependency graph. First, the test driver code is parsed to extract variable dependencies and construct a variable dependency graph. The vertices of the graph represent the code variables and the edges point from the dependent variables to the dependent variables. Next, using the set of variable dependencies as input, we obtain the variable dependency constraint relations, identifying each variable in the set and its constraint dependencies. Then, constraint variables are extracted and variable nodes are drawn. If dependencies exist between variables, edges are drawn between the variables and the type labels of the variable dependencies are extracted, along with the addition of variable constraint information. This process is repeated until the entire variable dependency graph is recursively traversed, ultimately resulting in the construction and generation of the constraint dependency graph.

Algorithm 1 Construction of Variable Constraint Dependency Graph

Input: $SourceCode, ConstrInfo$
Output: $VCDG$ //Variable Constraint Dependency Graph
VCDG = DiGraph()
NodeId = 0
procedure VISIT(*node*)
    PNode = f"node{*NodeId*}"
    Graph.node(*PNode*, *node.name*)
    NodeId = *NodeId* + 1
    CNode = f"node{*NodeId*}"
    for each *field*, *value* **in** iter_fields(*node*) **do**
        for each *item* **in** *value* **do**
            VCDG.edge(*PNode*, *CNode*, addLLMConstr(*item*, *PNode*, *CNode*, *ConstrInfo*))
        end for
        if isinstance(*value*, AST) **then**
            VCDG.node(*CNode*, *value.name*)
            VCDG.edge(*PNode*, *CNode*, addLLMConstr(*value*, *PNode*, *CNode*, *ConstrInfo*))
            visit(*value*)
        end if
    end for
end procedure
ASTTree = parse(*SourceCode*)
visit(*ASTTree*)

Constrained reasoning and backtracking

The design of constraint satisfaction is a difficult and challenging task. The development of artificial intelligence technology has further expanded the application domains of constraint reasoning, such as qualitative reasoning, task scheduling, and intelligent decision-making. Currently, there is ample room for further research and exploration in this field. Although LLM primarily focuses on NLP tasks, such as text generation and question-answering systems, its robust language understanding and generation capabilities can indirectly support constraint reasoning. By tapping into the potential of LLM in constraint reasoning, it is expected to enhance solution efficiency and accuracy.

First, constraint problems often involve a series of conditions and restrictions, which can be described in natural language. LLM can receive constraint conditions described in natural language and attempt to understand their meanings and logical relationships, thus facilitating the transformation of complex constraint problems into more comprehensible and manageable forms. Second, LLM provides cues and suggestions for constraint reasoning in the form of natural language generation. Based on its learned language data, it can generate texts related to constraint problems, which contain potential solutions or solution ideas. Further analysis of the generated texts can provide beneficial insights for constraint reasoning.

The input of constraint reasoning is a set of constraint conditions and problems to be solved, and the output is the solutions to the problems. LLM takes the current problem and preconditions as a set of constraints, transforming the problem solutions into solutions to several constraints. By traversing and analyzing the constraint dependency graph, the problem points are located based on the constraint information satisfied by the nodes and backtracks to the correct position. LLM re-reasons and solves based on the current constraints, iterates through test case code, and ultimately generates a reconstructed test case driver. If the reasoning process infers a step with a false value, it is called a conflict. In the case of a conflict, conflict backtracking can yield two results: backtracking to the initial assignment, indicating that the problem cannot be satisfied, and thus declaring "UNSAT"; otherwise, backtracking to a previous assignment and undoing all subsequent assignments. The aforementioned recursive backtracking algorithm has the following issues: on the one hand, when it encounters a conflict, it only knows that the current partial assignment will lead to a conflict and learns nothing else; on the other hand, it only backtracks one level at a time, potentially wasting significant time in a search space that is destined to fail.

We employ the conflict-driven clause learning (CDCL) algorithm to analyze and resolve conflicts by parsing and

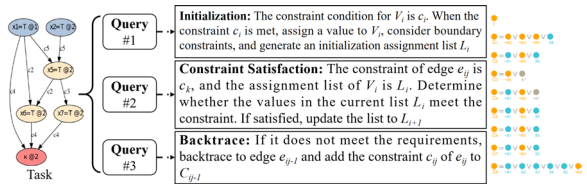


Fig. 7 Conflict analysis of the constraint dependency graph based on CDCL

traversing the path constraints on the constraint dependency graph, as illustrated in Fig. 7. It mainly includes three stages: Initialization, Constraint Satisfaction, and Backtrace. In the initialization phase, variables are assigned values under constraint conditions, generating an initial assignment list. During the constraint satisfaction determination stage, the current variable list is evaluated to check if it meets the constraint conditions. If the conditions are satisfied, the assignment list is updated; if not, the process backtracks. CDCL adds information on how to avoid previous conflicts resulting from a particular step as a new clause to the constraint list. Once this new constraint is satisfied, the previous conflict will no longer occur. LLM understands the semantic information of conflicts in variable constraints and generates potential conflict resolution strategies through mutation data, including suggestions for variable assignments, clause selection, or more complex backtracking strategies. By traversing the constraint dependency graph, analyzing constraint dependencies, and receiving strategic suggestions from the LLM, these suggestions are transformed into specific operations involving undoing decisions, reassigning values, or selecting clauses based on the suggestions and then continuing with the CDCL algorithm. When a conflict is detected, the process backtracks to the previous decision point regenerate new mutation data and attempts a new combination of decisions to find a path that resolves the conflict. After exceeding the set backtracking threshold, the current path is abandoned, and other new paths are explored until the entire dependency constraint graph has been traversed and all constraints are resolved. After each attempt to resolve a conflict, the CDCL algorithm provides feedback to the LLM, including the success or failure of the resolution. Based on this feedback, the LLM adjusts its generation strategies to gradually improve the capability and accuracy of conflict resolution.

Test case refactoring

Due to limited constraints, biases in training data, and incomplete code generation capabilities, the code generated by LLMs is prone to errors. These errors, often caused by syntax issues, logical errors, or environmental problems, are unsuitable for direct use as targets for automated testing. To obtain more constraints, we execute test case code and dynamically capture feedback results of program execution during runtime, including syntax errors, program output, and other relevant information. When encountering syntax errors, the interpreter prints errors and stack trace messages, assisting developers in locating the problem. This paper identifies and categorizes 11 typical types of syntax errors, including general SyntaxError, TypeError, ImportError, ValueError, and IOError, among others, such as spelling errors and indentation errors, and implements corresponding error capture and exception handling. Additionally, due to insufficient understanding of constraints, semantic errors may occur, such as algorithmic errors or inconsistencies between input and output with expected requirements. To address these issues, we propose a test case reconstruction scheme that employs a series of measures to adjust or correct erroneous content upon detecting syntax or semantic errors, aiming to maximize the accuracy of code execution results.

Figure 8 shows the test case reconstruction process adopted by LLM4TDG, leverages runtime syntactic and semantic issues encountered during test case execution as feedback to update the constraint dependency graph with incremental constraints, guiding the iterative optimization of test case code by LLM. First, the LLM generates and executes test cases based on data obtained from parsing the constraint dependency graph. Subsequently, feedback from the test case execution results guides the correction of syntax and semantic errors. During the reconstruction process, we incorporate a runtime monitoring and feedback mechanism. When the Python interpreter attempts to execute test case code, the runtime monitoring mechanism captures error feedback, including both syntactic and semantic errors. These error messages are then used as incremental constraints to update the constraint dependency graph, enabling the LLM to utilize its code repair capabilities to fix and iteratively reconstruct the test-driven code. While the test case reconstruction method using constraint feedback can resolve most issues, it cannot guarantee the generation of suitable test cases for all drivers. Therefore, we set a limited number of fault tolerance iterations for the test case reconstruction process. If this limit is exceeded, the reconstruction is abandoned. Although this approach may result in the loss of some correct test case data, it significantly improves the efficiency of batch test-driven code generation.

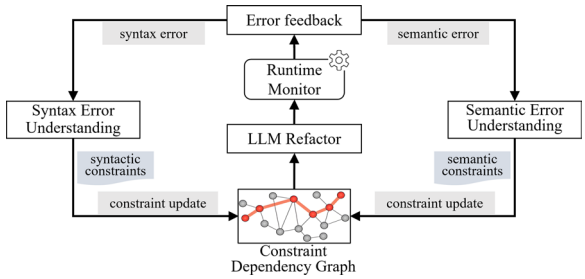


Fig. 8 Refactoring test cases based on constraint feedback

Evaluation

Experiment setting

The experimental hardware environment employed a 4U rack server equipped with four Nvidia A800 80 G GPUs, two Intel Gold 8358P 2.6 GHz 32-core 64-thread CPUs, 16 64 G DDR4 ECC REGS 3200 memory modules, 9.6T SATA SSD, and three 16T SATA mechanical hard drives. The experimental software environment utilized a Ubuntu 20.04 64-bit operating system server, Python 3.8.10 environment, with the installation of Pip 24.0, Setuptools 69.5.1, Wheel 0.43.0, and other tools. For model construction, open-source LLMs ran and provided inference services based on the experimental hardware environment, while for non-open-source LLMs, such as the GPT series, interactions were achieved through interface invocation.

In terms of model configuration, the LLM uses the Greedy Search approach for prediction, combined with a constraint reasoning mechanism to generate relatively deterministic samples for each task, thereby reducing the model's randomness. For instance, in the GPT-4 interface provided by OpenAI, the temperature parameter controls the similarity between the generated text and the given prompt. A higher temperature value results in more random output, whereas a lower temperature value makes the model's responses more deterministic and consistent. In the experiments, we set the temperature parameter to 0, ensuring that the LLM selects the token with the highest probability at each time step (t) when predicting the next token, thereby reducing the randomness in the LLM's responses.

Baselines

Aiming to validate the effectiveness of the proposed test-driven generation framework based on LLM constraint reasoning, this paper employed two categories of SOTA LLMs in general domains and code generation domains, as shown in Table 2. These include four LLMs in general domains and five specialized LLMs in code generation domains. The same model parameter settings were used as the test baseline for evaluation and comparison in the experiments.

Dataset

The experimental dataset is sourced from two parts: EvalPlus and pypi_malregistry. EvalPlus is a rigorous test code generation evaluation framework that has extensively assessed the code generation capabilities of various LLMs, as shown in Table 3. By automatically generating test inputs, EvalPlus extends the existing HUMANEVAL code test benchmark and creates the HUMANEVAL+ test dataset. Cross-validation is used to check the correctness of the code generation functionality. Among them, HUMANEVAL is one of the pioneering and most widely studied manually written benchmarks for code synthesis based on LLMs. Each task has a reference solution consisting of 164 pairs of Python function signatures with docstrings and related test cases for correctness checking. In HUMANEVAL+, there are an average of 764.1 test cases per task, while HumanEval has only 9.6 test cases per task on average. Even the minimized HUMANEVAL+-MINI has 16.1 test cases per task. For each task, EvalPlus automatically generates high-quality test inputs based on mutation methods, running

Table 2 Overview of Evaluated Baselines Models

Type	Module	Manufacturer	Parameter	Tokens	Open	Release
General	GPT-4	OpenAI	N/A	32K	×	2023
	Mixtral:8x22b-instruct	Mixtral AI	176B	65K	✓	2024
	Llama3-70B-Instruct	Meta	70B	8K	✓	2024
	Wizardlm2:8x22b	Microsoft	176B	64K	✓	2024
	Qwen-72b	Alibaba	72B	32K	✓	2023
Coding	DeepSeek-Coder-33B	DeepSeek	33B	16K	✓	2023
	CodeQwen:7b-chat	Alibaba	7B	8K	✓	2023
	CodeLlama-70b	Meta	70B	100K	✓	2024
	Wizardcoder:33b-v1.1	WizardLM	33B	32K	✓	2024

Table 3 Comparison of Dataset Types and Test Cases

Dataset Type	Tasks	Test Cases			
		Avg.	Med.	Min.	Max.
HumanEval		9.6	7.0	1	10^2
HumanEval$^+$	164	764.1	982.5	12	1100
HumanEval$^+$-Mini		16.1	13.0	5	110

type-aware mutations to generate 1,000 additional inputs, increasing the original HUMANEVAL test case size by nearly 80 times. This ability to identify a large number of undetected error codes generated by LLMs demonstrates the effectiveness of HUMANEVAL+ as a benchmark for accurate code generation evaluation in the code generation domain.

To validate the applicability and generalization capabilities in real-world scenarios, the experiment selected the Python third-party dependency library dataset, pypi_malregistry (Guo W. et al. 2023), in the context of security testing tasks, as shown in Table 4. After manual collection and confirmation, the pypi_malregistry dataset consists of Python third-party libraries artificially injected with malicious code. In addition to samples that acquire sensitive user information and execute commands, this dataset also includes samples that employ advanced techniques such as code encryption and obfuscation, as well as indirect package imports, to evade security detection, thus increasing the difficulty of detecting malicious behavior. Unlike executable files, third-party dependency libraries cannot automatically run code; malicious code can only be discovered through static analysis. To employ dynamic methods to trigger malicious behavior, test cases must be written to drive the execution of the libraries. By providing task requirements to the LLM4TDG to generate test cases, dynamic detection can determine whether the dependency libraries contain triggerable malicious behavior. Since this dataset does not inherently include task descriptions related to code security testing, the application scenario for generating test case code is more generalizable.

Due to the presence of dependency library files in the dataset that do not conform to the Python package installation format and some libraries that, once installed, function as command-line tools rather than generating test cases through API calls in Python code, these instances fall outside the scope of what the proposed method can address. Additionally, considering the complexity of real-world programs, the experiment focused on the security testing of the widely used Python library requests. After de-duplicating attack patterns from the compromised request libraries and their variant samples, five typical libraries with different attack patterns were selected as the test objects. The Radon (2024) tool was used to assess the complexity of the target objects, each having more than 6,500 lines of code, over 630 functions, and an average cyclomatic complexity of 2.0. Table 4 details the specific test samples. These samples differ in attack methods and the locations where the malicious code is triggered. Furthermore, Wireshark is used for packet capture to analyze whether the libraries contained abnormal network connections, thereby determining the presence of malicious behavior.

Metrics

To evaluate the effectiveness of LLM4TDG, we define evaluation metrics from three different dimensions, namely, constraint reasoning effectiveness, error correction capability, and functional correctness, reflecting the comprehensive capabilities of the framework.

Constraint Reasoning Effectiveness To assess the impact of constraint guidance on reasoning effectiveness, this paper defines the constraint gain ratio (*CGR*) metric. The *CGR* metric represents the ratio of the number of tasks where the average constraint satisfaction rate of LLMs exceeds that of cases without the constraint dependency graph to the total number of tasks where the constraint dependencies are not fully satisfied without the use of the constraint dependency graph, as shown in Eq. 2. Here, n represents the total number of tasks, x_i represents the number of tasks where the average constraint satisfaction rate of different LLMs is better when using the constraint dependency graph than when not using it, and y_i represents the number of tasks where the constraint dependencies are not fully satisfied without using the constraint dependency graph.

Table 4 Overview of Evaluated Baselines Models

Project	Version	NLOC	FunCnt	AvgCCN	Malicious File	Code Line
pyquest	2.28.3	7076	641	2.0	exceptions.py	46
requests-httpx	2.28.1	7079	641	2.0	__version__.py	4
reequests	2.27.1	6512	631	2.0	__init__.py	181
pyiopcs	2.28.1	7077	641	2.0	models.py	9
url-requests	3.0.0	7127	641	2.0	setup.py	173

$$CGR = \frac{\sum_{i=1}^{n} x_i}{n - \sum_{i=1}^{n} y_i} \times 100\%. \quad (2)$$

Error Fixing Capability A test driver is considered effective if the generated test case code has no syntax errors and satisfies the known semantic constraints during runtime. To evaluate the impact of the constraint dependency analysis method on improving the error-fixing capability (*EFC*) of test drivers, this paper adopts the net problem-solving rate (*NPR*) and iteration efficiency rate (*IER*) as evaluation metrics in an effective test driver generation process. *NPR* represents the ratio of the net value of solved problems to the total number of problems accumulated during the iteration process, as shown in Eq. 3. Here, n represents the initial number of problems, m represents the number of iterations, x_i represents the number of new problems generated in each iteration, and y represents the number of solved problems.

$$NPR = \left(y - \sum_{i=1}^{m} x_i \right) / \left(n + \sum_{i=1}^{m} x_i \right) \times 100\%. \quad (3)$$

The iteration efficiency rate (*IER*) represents the ratio of the remaining number of iterations to the iteration threshold, as shown in Eq. 4. Here, T represents the iteration threshold, and m represents the number of iterations.

$$IER = \frac{T - m}{T} \times 100\%. \quad (4)$$

The error fixing capability (*EFC*) comprehensively considers both the net problem-solving rate (*NPR*) and the iteration efficiency rate (*IER*) factors through a weighted summation, which represents the overall program repair capability, as shown in Eq. 5.

$$EFC = \left(NPR \times \omega_i + IER \times \omega_j \right) \times 100\%. \quad (5)$$

Functional Correctness The current mainstream method for evaluating the functional correctness of code generation is the modified *pass@k* method. In the *pass@k* method, k represents the number of sample solutions generated for each problem, and if any one of the k solutions passes the unit test, the problem has been solved. However, the variance of the *pass@k* evaluation method can be relatively large. The improved *pass@k* method generates several sample solutions, counts the number of solutions that can pass the unit test, and then calculates the unbiased estimate of the pass rate, as shown in Eq. 6. Here, c represents the number of solutions that pass the unit test, n represents the total number of answers generated for each problem, and k is a constant array. The value of the improved *pass@k* is determined based on the value of n and does not require a dynamic setting.

$$pass@k = \underset{\text{Problems}}{E} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]. \quad (6)$$

In the following section, we comprehensively evaluate the effectiveness of the LLM4TDG framework in terms of constraint reasoning effectiveness, error-fixing capability, and functional correctness of the generated code using the proposed test benchmark, test evaluation dataset, and evaluation metrics.

Constraint reasoning effectiveness

The experiments obtained 164 test tasks from the EvalPlus dataset. Through the LLMs, the semantic understanding of interface testing requirement descriptions, API call examples, return value descriptions, and other information was obtained to extract constraints and construct constraint dependency graphs. Reasoning was conducted on the schemes to satisfy constraints for different tasks. By setting up a controlled experiment, we compared the constraint satisfaction situation by adding experiments that directly generated test case code tasks without using constraints. The statistical data of the average constraint satisfaction rates of different LLMs for different test case generation tasks, with and without constraints, are shown in Fig. 9. In 164 test case generation tasks, for each task, the pass rate of all models on the test cases is first calculated separately for cases with and without the use of constraint reasoning. Then, calculate the incremental pass rate obtained by each model using constraint inference.

The orange section reflects the increase in constraint satisfaction achieved through constraint dependency. Without using constraint dependency reasoning, 17 tasks already fully satisfied the constraint requirements and thus did not require additional constraint satisfaction gains. After excluding these tasks, the statistical data on the test pass rates with and without using constraints indicate that using constraint reasoning for test case generation of tasks with different difficulties can provide more effective solutions, leading to a higher constraint satisfaction of 70 additional tasks out of 147, accounting for a constraint gain ratio of 47.62%. Furthermore, we counted the number of constraints passed by LLMs in all test case generation tasks, as shown in Fig. 10. After completing all test case generation tasks, the number of effectively resolved problems without the proposed constraint reasoning method was 918, whereas with the constraint reasoning method, the number increased to 1008.

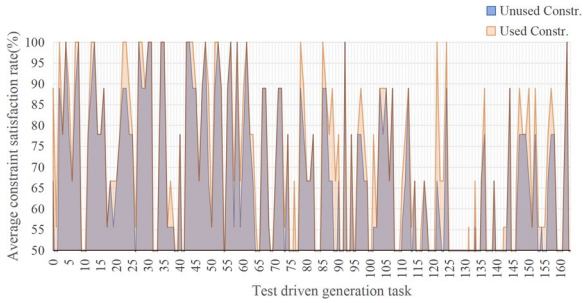


Fig. 9 The test-driven generation task constraint satisfies the validity

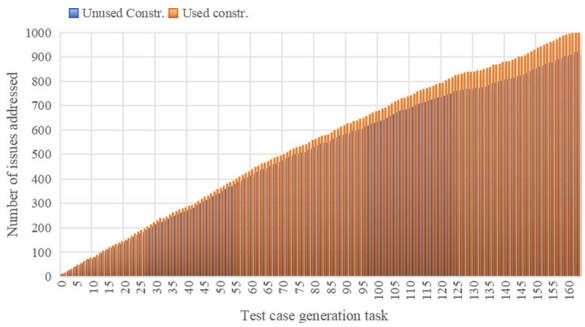


Fig. 10 Statistics on the number of testing task constraints passed by LLMs

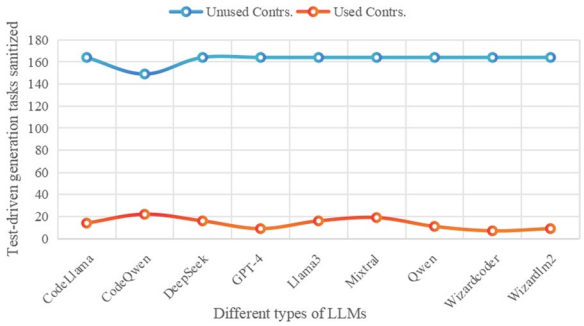


Fig. 11 A comparative analysis of the number of test-driven program sanitizations

The number of constraints passed when using constraint reasoning was significantly greater than when constraints were not used. We analyzed the reasons for the insignificant constraint satisfaction in some cases. Part of the reason is that the complexity of the task requirement descriptions exceeded the LLMs' ability to understand constraints, thus preventing the construction of effective constraint dependencies. Additionally, although LLMs were required to generate data according to formatting specifications, some LLMs' outputs did not fully meet the formatting requirements, resulting in issues in the process of output data and affecting the LLMs' ability to construct constraint dependencies.

Error fixing capability

The types of errors in test drivers include syntax errors and semantic errors. If the driver can successfully run without any syntax error messages, the test driver generated by the LLM is syntactically correct. On the basis of syntactic correctness, if the test driver satisfies the given semantic constraints, the test driver generated by the LLM is semantically correct. When the program encounters syntax or semantic errors, the LLM4TDG framework performs program repair based on specific error feedback information. Figure 11 presents a comparative analysis of the number of test-driven program sanitizations, both with and without the employment of constraint dependencies. In all test case generation tasks, GPT-4 required the purification of all 164 test cases when constraint reasoning was not used, whereas only 14 cases needed purification when constraint reasoning was applied. The results indicate that when constraint dependencies are incorporated, the number of programs requiring sanitization is significantly lower compared to scenarios where constraint dependencies are not utilized. This result demonstrates that when constraint dependencies are effectively applied, the program contains fewer error codes.

The experiment evaluated the error-fixing capabilities of nine LLMs from both general-purpose and code generation-specific domains on 164 code generation tasks from the EvalPlus dataset, focusing on syntax and semantic errors, as shown in Table 5. According to the comprehensive evaluation metric EFC, GPT-4 from the general-purpose domain achieved the highest score of 87.26% in fixing syntax and semantic errors, surpassing models such as DeepSeek-Coder and CodeLlama, which specialize in code generation. In contrast, the Qwen group had the lowest EFC score (28.42%) for fixing syntax and semantic errors. We also separately analyzed the error-fixing performance of different LLMs for syntax and semantic errors. Fig. 12 a illustrates the correction statistics for syntax issues across different LLMs. GPT-4 successfully resolved 78 syntax issues, while 14 syntax issues remained unresolved. In comparison, Qwen correctly fixed 118 syntax issues, but 376 syntax issues were not successfully corrected. In terms of syntax error fixing, GPT-4 demonstrated higher capabilities, resulting in fewer syntax errors and a relatively lower total number of fixes. However, Qwen encountered more errors during syntax error fixing, leading to more fixes and lower fixing performance. Fig. 12b presents the correction statistics for semantic issues across different LLMs. GPT-4 successfully addressed 67 semantic issues, with

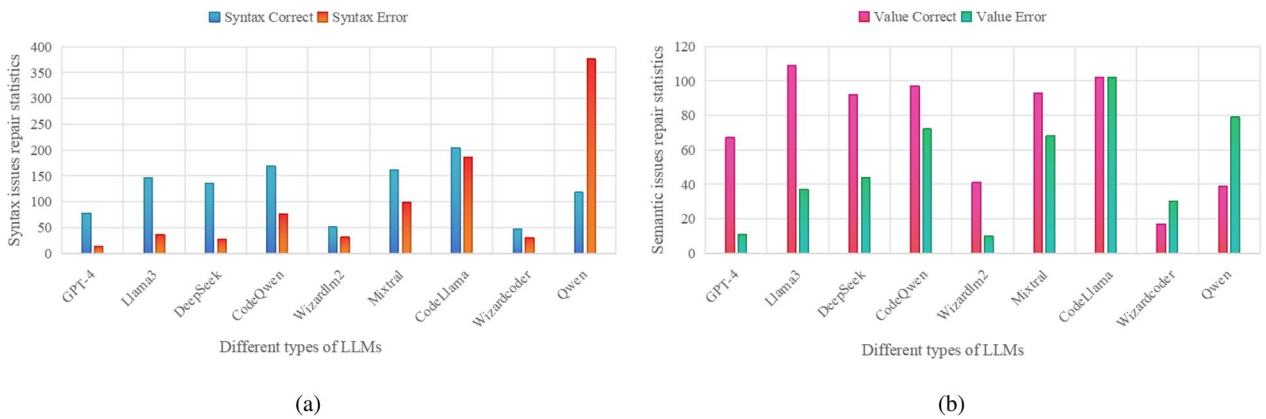
Table 5 Comparison of repair capabilities for different types of LLMs in problem resolution

LLM	Syntax Problem		Value Problem		Repair	Repair	NPR(%)	IER(%)	EFC(%)
	Correct	Error	Correct	Error	Limit	Iterator			
GPT-4	78	14	67	11	390	42	85.29	89.23	87.26
Llama3-Instruct	146	36	109	37	730	117	77.74	83.97	80.86
DeepSeek-Coder	136	27	92	44	690	96	76.25	86.09	81.17
CodeQwen-chat	169	76	97	72	870	154	64.25	82.30	73.75
Wizardlm2	51	31	41	10	255	65	69.17	74.51	71.84
Mixtral-instruct	161	99	93	68	965	179	60.33	81.45	70.89
CodeLlama	204	186	102	102	1075	232	51.52	78.42	64.97
Wizardcoder	47	30	17	30	235	80	61.96	65.96	58.79
Qwen	118	376	39	79	590	406	25.65	31.19	28.42

11 unresolved, whereas Llama3 corrected 109 semantic issues, leaving 37 unresolved. For semantic error fixing, GPT-4 still exhibited more correct and fewer incorrect fixes, reflecting its excellent semantic error-fixing ability. Comparatively, CodeLlama, Wizardcoder, and Qwen performed poorly, with some cases even showing an increase in errors after fixation. Additionally, the paper revealed that models from the code generation domain performed less effectively in terms of code generation repair than general-purpose models. We analyzed the reasons for this, suggesting that code generation models focus primarily on learning code-related tasks but lack the ability to capture and understand syntax and semantic error information, which cannot be translated into effective syntactic and semantic constraints. This limitation restricts their ability to fix syntax and semantic errors.

Functional correctness

Although LLM4TDG is capable of fixing erroneous code and satisfying known syntactic and semantic constraints, this does not equate to the functional correctness of the generated code. Limited constraints or the existence of complex and difficult-to-understand task requirements may lead to undetectable errors in the test case code generated by LLMs. In this paper, we employ the EvalPlus evaluation method, which utilizes an unbiased pass@k approach to assess the correctness of synthetic code generated by different LLMs on a given dataset. The EvalPlus evaluation method is based on type-aware fuzz testing that mutates input data. It derives a set of seed inputs from the LLM, which are used to initialize a seed pool and guide the generation process. At each iteration, an input is randomly selected from the seed pool for mutation, and new inputs that satisfy the program constraints are added to the seed pool, initiating a repeated generation process. Type-aware input mutation, built upon high-quality seed inputs generated by LLMs, can produce a vast amount of input data for comprehensively evaluating the functional correctness of SOTA LLM-synthesized code.



(a) (b)

Fig. 12 Statistics on correct and incorrect repairs relating to syntax and value issues

Table 6 Evaluation of functional correctness of test case generation by LLMs

LLM	pass@k without Constraints(%)		pass@k with Constraints(%)		Average	
	HUMAN	HUMAN+	HUMAN	HUMAN+	Improve(%)	
GPT-4	87.80	82.90	**92.16**	**87.14**	5.04	↑
Llama3-Instruct	75.00	70.10	81.70	74.40	7.58	↑
DeepSeek-Coder	75.00	70.10	79.36	73.74	5.51	↑
Wizardlm2	75.60	68.30	79.90	72.60	5.98	↑
CodeQwen-chat	64.00	59.80	76.20	70.70	18.66	↑
CodeLlama	54.30	48.20	61.00	54.90	13.07	↑
Mixtral-instruct	47.00	41.50	54.30	46.30	13.67	↑
Wizardcoder	43.90	39.60	46.36	42.04	5.87	↑
Qwen	37.20	29.30	43.30	35.40	18.35	↑
Average	62.20	56.64	68.25	61.91	**10.41**	↑

Under both scenarios with and without the utilization of LLM4TDG constraint dependencies, we conducted experiments to evaluate the functional correctness of different LLMs in testcase code generation, as presented in Table 6. The results show that employing LLM4TDG constraint dependencies significantly improves the average test pass rate of all LLMs by 10.41%. Specifically, the test pass rate of CodeQwen-chat is increased by 18.66%. Additionally, the LLM4TDG approach has surpassed the SOTA model GPT-4, reaching 92.16%, while the HUMANEVAL+ indicator attains 87.14%. Other models also achieve varying degrees of improvement in test pass rates, demonstrating the effectiveness of LLM4TDG in enhancing the functional correctness of testcase generation.

Real-world program effectiveness

Unlike the EvalPlus dataset, where test requirements are predefined, in general, TDD development, for scenarios where test case hints are absent, predefined test task descriptions need to be provided to LLM4TDG for generating general test cases. Different projects will have varying methods of establishing and describing test task requirements for testing real software projects. Our method can automatically generate executable test cases based on the test requirement descriptions for real Python projects of different complexities, which serve as inputs for LLM4TDG. For example, when conducting unit tests for one or multiple APIs in a project, it is necessary to clearly define the test case requirements for the API's function signatures, input data, and expected results. LLM4TDG understands these test requirements and transforms them into contextual constraints for test case generation, iteratively refining the test case code to meet the constraints outlined in the test task.

In the experiment, we used the code security detection scenario involving compromised code from the pypi_malregistry dataset of real Python dependency libraries as an example. The GPT-4 model, which performed best in functional correctness, was used, combined with dynamic sandboxing, to enhance the automated security testing of Python dependency libraries. A comparison was made between cases with and without constraints, and module ablation studies, along with performance overhead tests, were conducted to validate the effectiveness of our method. For the test case generation requirements in Python dependency library security detection, LLM4TDG transforms the test requirements into contextual constraints as a constraint dependency graph. By setting up the dependency library's installation environment and generating test case constraints based on the test requirements, the system automatically generates test drivers that satisfy the defined constraints using constraint reasoning and backtracking mechanisms.

Test requirement establishment

For the security detection scenario involving compromised Python third-party dependency libraries, it is necessary to define test requirements that reflect real-world conditions. This process involves manually establishing the requirement descriptions, which are then used as input for LLM4TDG. When attackers inject backdoors into a dependency library, they typically choose an injection point at a certain distance from the entry function, which is easy to trigger, making malicious code auditing and analysis more difficult. In the experiment, the dependency libraries' folders are first mapped to their corresponding package names, and the dir method is used to examine the APIs provided by the modules or

Table 7 Ranking of widely used request dependency library APIs according to GitHub statistics

Rank	API	Code	Repositories	Issues	Pull	Discussions	Commits	Wikis	Q	Valid
1	request	0.18	0.00	0.09	0.13	0.09	0.00	0.09	0.20	✓
2	Request	0.18	0.00	0.09	0.13	0.09	0.00	0.09	0.20	✓
3	get	0.00	0.19	0.00	0.00	0.00	0.12	0.00	0.22	✓
4	post	0.13	0.19	0.06	0.10	0.02	0.15	0.04	0.57	✓
5	Response	0.19	0.19	0.10	0.13	0.10	0.14	0.10	0.84	✓
6	Session	0.18	0.20	0.08	0.10	0.08	0.15	0.09	0.89	✓
6	session	0.18	0.20	0.08	0.10	0.08	0.15	0.09	0.89	×
8	api	0.20	0.18	0.10	0.15	0.10	0.14	0.10	0.93	×
9	put	0.19	0.20	0.10	0.15	0.10	0.14	0.09	0.94	✓
10	delete	0.19	0.20	0.10	0.15	0.10	0.15	0.10	0.94	✓

Table 8 Requests dependency library test case generation satisfiability conditions

API	Function Signature	Input	Output	Condition
request	requests.request(method, url, **kwargs)	method = 'POST', url = 'https://jsonplaceholder.typicode.com/posts', json={'title':"foo",'body':"bar",'userId':1}	{'title':'foo','body':'bar', 'userId':1,'id': 101}	IRC
Request	requests.Request(method, url, headers,...)	method = 'GET' url = 'https://jsonplaceholder.typicode.com/posts/1'	{"userId": 1, "id": 1, "title": "sunt ...", "body": "quia ..."}	IRC
get	requests.get(url, params, **kwargs)	url='https://jsonplaceholder.typicode.com/posts/1'	{"userId": 1, "id": 1, "title": "sunt ...", "body": "quia ..."}	IRC
post	requests.post(url, data, json, **kwargs)	url="https://jsonplaceholder.typicode.com/posts", data="title":"foo","body":"bar","userId":1	{'title':'foo', 'body': 'bar', 'userId':1, 'id': 101}	IRC
Response	requests.Response	N/A	N/A	NEE
Session	requests.Session	N/A	N/A	NEE
put	requests.put(url, data, **kwargs)	url='https://jsonplaceholder.typicode.com/posts/1', json="title": "foo", "body": "bar", "userId": 1	{'title':'foo', 'body': 'bar',userId':1,'id':1}	IRC
delete	requests.delete(url, **kwargs)	url='https://jsonplaceholder.typicode.com/posts/1'	status_code==200	IRC

objects. From GitHub, API interfaces and programming language types are used as filtering criteria, and the multi-criteria compromise solution ranking VIKOR (Tzeng et al. 2011) algorithm is employed to calculate the Q-value statistics of API usage for ranking. Widely used API interfaces are selected for testing. Table 7 shows the Top 10 API usage rankings for the requests library.

Since session and api are not defined in the requests documentation, invalid APIs were excluded. Next, the expert experience was used to clarify the inputs, outputs, and criteria for determining whether each API passes the functional correctness of the test cases. For each compromised requests dependency library, test cases include all API calls. Table 8 provides the API names, function signatures, input values, output results, and the criteria for determining whether the test case is passed for the requests library. In the determination criteria, IRC stands for 'Include Return Content', and NEE stands for 'No Error Execution.' When the compromised requests library passes all the test cases listed in the table, the library is considered to have successfully passed the test.

Evaluation of reasoning effectiveness

Table 9 shows the comparison of security test case results for the compromised requests library. By utilizing widely used requests APIs obtained from GitHub and manually establishing test requirements, the LLM4TDG framework was employed to validate the effectiveness of the tests, ensuring that the basic runtime environment was correct. Additionally, in cases where test cases were generated directly based on test requirements input into the LLM without constraint reasoning, a statistical analysis was conducted on whether the tested API was included, whether the test case was passed, whether malicious behavior was detected, and whether malicious behavior

Table 9 Comparison of the effectiveness of generating security test cases for requests contaminated libraries

Python Library			Prompt without Constraints				Prompt with Constraints			
Name	**Version**	**Package**	**Include API**	**Run Pass**	**Run Fail**	**Evil**	**Include API**	**Run Pass**	**Run Fail**	**Evil**
pyquest	2.28.3	pyquest	20	0	20	0	20	19	1	20
requests-httpx	2.28.1	requests	20	0	20	0	20	20	0	20
reequests	2.27.1	requests	20	0	20	20	20	20	0	20
pyiopcs	2.28.1	pyiopcs	20	0	20	0	20	19	1	20
url-requests	3.0.0	requests	20	0	20	20	20	19	1	20
All			100	0	100	40	100	97	3	100

existed. The experimental results show that when constraint reasoning was not used, although the generated responses could cover all APIs, the test code could not be executed correctly, leading to failure in passing the test cases. In contrast, when using the constraint reasoning mechanism, not only were all APIs covered, but nearly all test codes were executed correctly and passed the test cases. Therefore, using constrained reasoning is more effective than reasoning without the mechanism, both in terms of execution and in detecting malicious behavior.

Further, we analyzed the experimental results. Without constraint reasoning, the LLM could understand the testing intent based on the test requirements and generate results. However, it did not fully grasp the expected input and output values of the tests, leading to cases where the generated results were either non-executable or had input/output contents inconsistent with the requirements. In terms of malicious behavior detection, since the malicious behavior in the requests-httpx and url-requests samples is triggered not through API calls but during the execution of dependency installation files, malicious behavior was still triggered despite the test cases failing. Additionally, unlike requests-httpx, reequsts, and url-requests, which map to the known original package name of requests, pyquest and pyiopcs map to unknown package names, increasing the complexity for the LLM to understand the test requirements due to this difference in package name mapping. When using constraint reasoning, by applying constraints and iterative feedback on the expected input and output values, the system could pass the test cases and detect malicious behavior.

Ablation and cost study

In this section, we analyzed the impact of key components in the LLM4TDG framework on the overall system performance, exploring the interactions between different modules. The focus is on assessing the influence of the Basic Environmental Analysis module and the Test Driven Refactoring module on the test case generation module, as shown in Table 10. A statistical analysis of the performance overhead incurred using the LLM4TDG framework was conducted, with 'P' representing Pass and 'F' representing Fail. In the ablation study, when the Basic Environmental Analysis module was not used, none of the dependency libraries could import the required environments, making it impossible to proceed with the subsequent test case generation. However, all dependency environments were successfully imported when the Basic Environmental Analysis module was employed. Without the Test Driven Refactoring module, the generated test cases failed to meet the expected input and output requirements for all APIs, resulting in failed test runs. After incorporating the Test Driven Refactoring module, all test cases were successfully passed following multiple iterations.

Regarding computational resource overhead, the constraint reasoning and backtracking mechanism of the LLM4TDG framework is mainly reflected in the Test Driven Refactoring module. Since reasoning is performed via remote calls, it is not possible to directly measure hardware resources such as GPU and memory usage, but the computational resources and overhead required for the constraint reasoning and backtracking mechanism can be indirectly assessed through the number of input and output tokens as well as the time consumption. When the Test Driven Refactoring module was not used, the token consumption for generating test cases was 3,044, with a total time consumption of 88.29 s and an average test case generation time of 17.66 s. In contrast, when using the Test Driven Refactoring module, token consumption increased to 8,560, with a total time consumption of 271.37 s and an average test case generation time of 54.27 s. Compared to the case without the Test Driven Refactoring module, the LLM4TDG framework saw an increase of 181.21% in average token consumption and a 207.36% increase in average test case generation time.

Table 10 LLM4TDG framework module ablation and cost statistics

Python Library		Basic Environmental Analysis		Test Driven Refactoring					
		Not Used	Used	Not Used			Used		
Name	Version	Import	Import	Run	AvgTokens	AvgTime	Run	AvgTokens	AvgTime
pyquest	2.28.3	F	P	0/20	662	22.11	19/1	2615	90.86
requests-httpx	2.28.1	F	P	0/20	509	14.06	20/0	1213	38.31
reequests	2.27.1	F	P	0/20	540	14.4	20/0	1115	34.88
pyiopcs	2.28.1	F	P	0/20	755	20.92	19/1	2416	70.99
url-requests	3.0.0	F	P	0/20	578	16.80	19/1	1201	36.33
All		F	P	0/100	3044	88.29	97/3	8560	271.37

Discussion

Limitations. Although LLM4TDG enhances the understanding of testing requirements and improves the accuracy of test code generation, research on constraint reasoning in LLMs is still in the exploratory stage, and its effectiveness and performance are influenced by various factors, including model type, parameter size, dataset quality, and complexity of task requirement description. In addition, due to the introduction of a dynamic feedback mechanism in the framework, iterative code refactoring based on LLMs feedback results incurs additional time costs and, therefore, sacrifices some performance.

Future Work. LLM4TDG is not only suitable for automated testing but also for requirement-driven code generation, code repair, and low-code software development patterns. LLM4TDG can reduce human-computer interaction and reliance on expert knowledge. In the future, design-driven cross-modal software development and testing, such as user interface (UI) and UML software design, can be explored to empower LLMs to improve the efficiency and quality of automated development and testing throughout the software development lifecycle.

Conclusion

In this paper, we discuss the pertinent issue of inadequate software automation testing within the contemporary software development paradigm, particularly emphasizing component reuse and low-code patterns. We present a novel LLM-based constrained inference method for generating high-quality test drivers .

LLM4TDG leverages constraint reasoning and backtracking mechanisms to optimize the test driver code through iterative feedback, which automatically rectifies syntactical and semantic errors, ensuring the highest correctness of the generated test driver program. Compared to the previous work, the experimental results show that our framework significantly enhances LLMs' ability to interpret test objective constraints, leading to improved error correction and functional accuracy during test driver code generation. Besides, we validate the effectiveness of LLM4TDG's generalization and scalability capabilities in conjunction with real-world application scenarios.

Acknowledgements
We would like to thank the anonymous reviewers and editors for detailed comments and useful feedback.

Author Contributions
Jingqiang Liu participated in all the work, designed the framework and experiments, and wrote the manuscript. Ruigang Liang, Yue Zhang and Xiaoxi Zhu provided suggestions on the framework design and joined the discussion of this work. Ruigang Liang, Yuling Liu and Qixu Liu reviewed the manuscript and gave suggestions on the revision of the details of the article. All authors read and approved the final manuscript.

Funding
This work is supported by the Youth Innovation Promotion Association CAS, the Key Laboratory of Network Assessment Technology at Chinese Academy of Sciences, Beijing Key Laboratory of Network Security and Protection Technology, the National Key Research, and science and technology project of the headquarters of State Grid Corporation of China (Research and Application of Network Security Situation Awareness Technology Based on Knowledge Graph, 5700-202352606A-3-2-ZN).

Availability of data and materials
All datasets used are public datasets.

Declarations

Competing interests
The authors have no Conflict of interest to declare that are relevant to the content of this article.

Received: 24 June 2024 Accepted: 4 November 2024
Published online: 15 May 2025

References

Aniculaesei A, Vorwald A, Rausch A (2019) Automated generation of requirements-based test cases for an automotive function using the scade

toolchain. In: 11th international conference on adaptive and self-adaptive systems and applications (ADAPTIVE 2019), p 69–74
Aniculaesei A, Howar F, Denecke P, Rausch A (2018) Automated generation of requirements-based test cases for an adaptive cruise control system. In: 2018 IEEE workshop on validation, analysis and evolution of software tests (VST), IEEE, p 11–15
Austin J, Odena A, Nye M, Bosma M, Michalewski H, Dohan D, Jiang E, Cai C, Terry M, Le Q, et al. (2021) Program synthesis with large language models. arXiv preprint arXiv:2108.07732 . https://doi.org/10.48550/arXiv.2108.07732
Baliga N (2017) A deep dive into a data-driven world of test. In: 2017 IEEE AUTOTESTCON, IEEE p 1–8
Bang S, Nam S, Chun I, Jhoo HY, Lee J (2022) Smt-based translation validation for machine learning compiler. In: International conference on computer aided verification, Springer, p 386–407
Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
Carlos F, Papadakis M, Durelli V, Delamaro EM (2014) Test data generation techniques for mutation testing: a systematic mapping. In: Workshop on experimental software engineering (ESELAW'14), p 419–432
Cassano F, Gouwar J, Nguyen D, Nguyen S, Phipps-Costin L, Pinckney D, Yee M-H, Zi Y, Anderson CJ, Feldman MQea (2023) Multipl-e: a scalable and polyglot approach to benchmarking neural code generation. IEEE Transactions on Software Engineering
Chen M, Tworek J, Jun H, Yuan Q, d Pinto O, HP, Kaplan J, Edwards H, Burda Y, Joseph N, et al, GB (2021) Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374
Deng Y, Xia CS, Yang C, Zhang SD, Yang S, Zhang L (2023) Large language models are edge-case fuzzers: testing deep learning libraries via fuzzgpt. arXiv preprint arXiv:2304.02014
Di Nardo D, Pastore F, Briand L (2015) Generating complex and faulty test data through model-based mutation analysis. In: 2015 IEEE 8th international conference on software testing, verification and validation (ICST), IEEE, p 1–10
Divakaran M, Peddinti DT, LLMs S (2024) LLMs for cyber security: new opportunities. 2404.11338
Eichenbaum B (1965) The theory of the formal method. vol. 6
Elman JL (1990) Finding structure in time. Cognitive Sci 14(2):179–211
Fried D, Aghajanyan A, Lin J, Wang S, Wallace E, Shi F, Zhong R, Yih S, Zettlemoyer L, Lewis M (2023) Incoder: a generative model for code infilling and synthesis. In: The eleventh international conference on learning representations
Green CC (1981) Applications of theorem proving to problem solving. Proc. of IJCAI-69
Gulwani S (2011) Automating string processing in spreadsheets using input-output examples. Sigplan Not 46(1):317–330
Gulwani S, Polozov O, Singh R (2017) Program synthesis. Found Trends Program Lang 4(1–2):1–119
Guo W, Xu Z, Liu C, Huang C, Fang Y, Liu Y (2023) An empirical study of malicious code in PyPI ecosystem. 2309.11021. https://arxiv.org/abs/2309.11021
Holler C, Herzig K, Zeller A (2021) Fuzzing with code fragments. In: 21st USENIX security symposium (USENIX Security 12), USENIX Association, Bellevue, WA, p 445–458
Hsu I-C, Ting D-H, Hsueh N-L (2014) Mda-based visual modeling approach for resources link relationships using uml profile. Comput Stand Interfaces 36(3):648–656
Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan K (2023) Swe-bench: can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770
Kalyan A, Mohta A, Polozov O, Batra D, Jain P, Gulwani S (2018) Neural-guided deductive search for real-time program synthesis from examples. In: International conference on learning representations
Kannavara R, Havlicek CJ, Chen Bea (2015) Challenges and opportunities with concolic testing. In: 2015 national aerospace and electronics conference (NAECON), p 374–378
King JC (1976) Symbolic execution and program testing. Commun ACM 19(7):385–394
Li Y, Choi D, Chung J, Kushman N, Schrittwieser J, Leblond R, Eccles T, Keeling J, Gimeno F, Aea Dal Lago (2022) Competition-level code generation with alphacode. Science 378(6624):1092–1097
Liu X, Xu G, Hu L, Fu X, Dong Y (2011) An approach for constraint-based test data generation in mutation testing. J Comput Res Develop 48(4):617–626
López-Ibáñez M, Stützle T, Dorigo M (2015) Ant colony optimization: a component-wise overview
Lyu Y, Xie Y, Chen P, Chen H (2023) Prompt fuzzing for fuzz driver generation. arXiv preprint arXiv:2312.17677
Minj B (2015) Generation of test cases based on analysis of simulink stateflow models. Int J Comput Appl 975:8887
Moitra A, Siu K, Crapo AW et al (2019) Automating requirements analysis and test case generation. Requir Eng 24(3):341–364
Mussa M, et al, SO (2009) A survey of model-driven testing techniques. In: Ninth international conference on quality software, p 167–172
Nijkamp E, Pang B, Hayashi H, Tu L, Wang H, Zhou Y, Savarese S, Xiong C (2023) Codegen: an open large language model for code with multi-turn program synthesis. In: The eleventh international conference on learning representations
Offutt AJ, Jin Z, Pan J (1999) The dynamic domain reduction procedure for test data generation. Softw Pract Exp 29(2):167–193
Openwall: Follow @Openwall on Twitter for new release announcements and other news (2024). https://www.openwall.com/lists/oss-security/2024/03/29/4. Accessed 05 Apr 2024
Papineni K, Roukos S, Ward T, Zhu W-J (2002) Bleu: a method for automatic evaluation of machine translation. In: Proceedings of the 40th annual meeting of the association for computational linguistics, p 311–318
Pargas R, Harrold MJ, Peck R (1999) Test-data generation using genetic algorithms. Softw Test Verification Reliab 9(4):263–282
PyPI: radon (2024). https://pypi.org/project/radon. Accessed 5 Sep 2024
Radford A, Narasimhan K, et al (2018) TS: Improving language understanding by generative pre-training. OpenAI
Rani S, Dhawan H, Nagpal G, Suri B (2019) Implementing time-bounded automatic test data generation approach based on search-based mutation testing. In: Progress in advanced computing and intelligent engineering: proceedings of ICACIE 2017, Vol 2. Springer, p 113–122
Schneider S (2021) Using model-based testing for creating behaviour-driven tests. PhD thesis, Wien
Sonatype: 8th annual state of the software supply chain (2022). https://www.sonatype.com/state-of-the-software-supply-chain/open-source-dependency-management-trends-and-recommendations. Accessed 16 Feb 2024-02-16
Synopsys: Open Source Security and Risk Analysis 2023. https://www.gcomtw.com/mailshot/Synopsys/2303BlackDuck/repossra2023ch.pdf. Accessed 11 Feb 2024
Tzeng GH, Huang JJ (2011) Multiple attributes decision making - methods and applications. Lecture Notes in Economics Mathematical Systems 404(4):287–288
Wagner F, Schmuki R, Wagner T, Wolstenholme P (2006) Modeling Software with Finite State Machines: A Practical Approach, New York. Auerbach Publications
Wang L, Li X, Zheng G (2005) Research on model-driven software testing. Computer Science
Whalen MW, Rajan A, Heimdahl MPEea (2006) Coverage metrics for requirements-based testing. In: Proceedings of the 2006 international symposium on software testing and analysis, pp. 25–36
Winterer D, Zhang C, Su Z (2020) On the unusual effectiveness of type-aware operator mutations for testing smt solvers. Proceedings of the ACM on programming languages 4(OOPSLA), pp. 1–25
Xia CS, Paltenghi M, Tian JL, Pradel M, Zhang L (2024) Universal fuzzing via large language models. In: 46th international conference on software engineering (ICSE)
Xia CS, Wei Y, Zhang L (2023) Automated program repair in the era of large pre-trained language models. In: 2023 IEEE/ACM 45th international conference on software engineering (ICSE), IEEE, pp. 1482–1494
Xu FF, Alon U, Neubig G, Hellendoorn VJ (2022) A systematic evaluation of large language models of code. In: Proceedings of the 6th ACM SIGPLAN international symposium on machine programming, pp. 1–10

Xuanjing supply chain security intelligence center: Xuanjing supply chain security intelligence center (2024). https://www.freebuf.com/news/398129.html. Accessed 25 Apr 2024

Yang B, Wu J, Xu L, Bi K, Liu C (2014) A method for modeling software testing requirements and generating test cases. Chinese J Comput 37(3):17. https://doi.org/10.3969/j.issn.1672-528X.2014.11.016

Yao X, Gong D, Li B (2016) Evolutionary generation of path coverage test data with neural networks. J Softw 27(04):828–838

Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman Sea (2018) Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 conference on empirical methods in natural language processing

Zhang C, Bai M, Zheng Y, Li Y, Xie X, Li Y, Ma W, Sun L, Liu Y (2023) Understanding large language model based fuzz driver generation. arXiv preprint arXiv:2307.12469

Zheng Q, Xia X, Zou X, Dong Y, Wang S, Xue Y, Wang Z, Shen L, Wang A, Li Yea (2023) Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. arXiv preprint arXiv:2303.17568

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.